ACCOUNT SERVICE · CODE REVIEW WALKTHROUGH · 2026.09.29

금융 코드 리뷰 R01–R19 코드로 보는 문제와 수정

`account_financial_code_review_2026-09-29.pdf`의 지적 사항을 실제 코드와 함께 다시 풀어 쓴 문서입니다. 각 항목에서 **무엇이 문제였고, 코드를 어떻게 바꿨고, 무엇으로 검증했는지**를 이 문서만으로 확인할 수 있습니다.

항목	내용
리뷰 기준 커밋	`7d6f766` (IS-28 정산 배치까지)
수정 상태	작업 트리에 반영됨 · 아직 커밋 전
Before 코드	`git show 7d6f766:<경로>`
After 코드	현재 작업 트리
테스트	35개 클래스 / 134개 통과 (MySQL·Redis Testcontainers, `HARDENING_REPORT.md` 기준)

문제 재현된 상황 **BEFORE** 기준 커밋 코드 **AFTER** 수정된 코드 **검증** 회귀 테스트

코드 블록은 이해에 필요한 부분만 발췌했으며 생략은 `// ...`로 표시했습니다. 이 문서는 엔지니어링 검토 자료이며 금융 규제 준수 인증을 의미하지 않습니다.

목차

0. 한눈에 보기

1. 요청 흐름 — 무엇이 바뀌었나

2. 데이터 안전 (R01~R04)

- **R01** 거래 커밋과 멱등성 완료 저장이 분리됨 → 중복 입금
- **R02** 멱등성 키에 사용자·작업·요청 내용이 묶이지 않음
- **R03** 금액 소수 자릿수·상한 검증 없음 → 총액 보존 위반
- **R04** 일반 사용자가 입금 API로 잔액을 만들어 냄

3. 보안·배포 기본 (R05~R09)

- **R05** 검증 실패 로그에 비밀번호 평문
- **R06** `X-Forwarded-For`를 그대로 믿어 로그인 제한 우회
- **R07** Redis 연결 누락 + 장애 시 제한 해제(fail-open)
- **R08** 운영 스키마를 앱 기동 시 자동 변경 (`ddl-auto: update`)
- **R09** Ingress에 HTTPS 강제 구성이 없음

4. 운영 신뢰 (R10~R19)

- **R10** 커밋 전 성공 로그 + 감사 연결 정보 부족
- **R11** 대사 배치의 `businessDate`가 실제 집계 기준과 무관
- **R12** 대사 불일치가 로그로만 남음
- **R13** 배치 누락·중단 후 복구 경로 없음
- **R14** 테스트를 통과하지 않은 커밋도 배포됨
- **R15** `latest` 적용 후 SHA로 바꾸는 이중 롤아웃 + 동시 배포
- **R16** 관리 포트만 헬스체크 → 실제 API 장애를 놓침
- **R17** 동시 이체 테스트가 전부 실패해도 통과
- **R18** 멱등성 헤더의 빈 값·길이·충돌 종류 미검증
- **R19** 거래 이력 페이지 정렬 기준이 고유하지 않음

5. 차단 이슈 외 함께 고친 것 (리뷰 PDF 10장)

6. 아직 남은 것 (이번 수정 범위 밖)

7. API 호환성 주의

0. 한눈에 보기

ID	등급	한 줄 문제	한 줄 수정	핵심 파일
R01	P1	거래는 커밋됐는데 멱등성 키가 지워져 재시도 시 **돈이 두 번** 들어감	키 선점·잔액·이력·완료 저장을 **한 트랜잭션**으로	`IdempotencyService`
R02	P1	남의 키로 **남의 거래 응답**을 받을 수 있음, 금액 바꿔도 "성공"	키를 `(사용자, 키)` 로 유일화 + 요청 지문(SHA-256) 비교	`IdempotencyKey`, `IdempotencyService`
R03	P1	0.015원 이체 시 DB 반올림으로 **총액이 1.00→1.01**	소수 2자리·상한 검증, 반올림 대신 거절	`MoneyAmount`, DTO `@Digits`
R04	P1	아무나 자기 계좌에 **돈을 찍어낼 수** 있음 (입금 API)	모의 입금 기본 OFF, prod 강제 차단	`TransactionService`
R05	P1	검증 실패 로그에 **비밀번호 평문**	로그엔 코드·카테고리만	`GlobalExceptionHandler`
R06	P1	`X-Forwarded-For` 위조로 로그인 제한 **우회**	`remoteAddr` 만 사용	`RateLimitFilter`
R07	P1	배포에 Redis 주소 없음 + 장애 시 **제한 해제(fail-open)**	Redis 주소 필수, 장애 시 503 (fail-closed)	`TokenBucketRateLimiter`
R08	P1	운영에서도 `ddl-auto: update`	Flyway V1~V4 + prod `validate`	`db/migration`, `DatabaseMigration`
R09	P2	Ingress에 TLS 없음	websecure + TLS Secret + NetworkPolicy	`k8s/ingress.yaml`
R10	P2	커밋 **전에** "이체 완료" 로그, 거래 추적 ID 없음	`afterCommit` 로그 + 이력에 actor/키 연결	`TransactionService.record()`
R11	P2	`businessDate`가 실제 집계 기준과 무관	"현재 원장 정합성 검사"로 의미 명확화	`BalanceReconciliation*`
R12	P2	불일치가 WARN 로그로만 남음	결과 테이블 저장 + 지표 + 경보	`ReconciliationResults`
R13	P2	01시에 앱이 죽어 있으면 그날 대사 **누락**	5분 폴링 + 수동 복구 스크립트	`BalanceReconciliationScheduler`
R14	P2	테스트 안 거친 커밋도 배포됨	CD가 CI 성공에 `needs` 의존	`cd.yml`
R15	P2	`latest` 적용 후 SHA로 바꾸는 이중 롤아웃	digest 하나로 렌더링해 1회 적용 + 직렬화	`cd.yml`, `render_manifests.py`
R16	P2	관리 포트(9090)만 헬스체크	주 포트 `/livez` `/readyz`	`k8s/deployment.yaml`
R17	P2	이체가 **전부 실패해도** 동시성 테스트 통과	성공 건수·이력 수까지 검증	`TransactionConcurrencyTest`

ID	등급	한 줄 문제	한 줄 수정	핵심 파일
R18	P2	빈 키·초과 길이 키 검증 없음	정규식 검증 + 제약 이름으로 충돌 판별	`IdempotencyService`
R19	P2	`createdAt` 동률 시 페이지 순서 흔들림	`id DESC` 고정 + 커서 API	`TransactionHistoryRepository`

1. 요청 흐름 — 무엇이 바뀌었나

Before

```
Controller
  └ IdempotencyService.execute(key, ...)          ← 트랜잭션 없음
        ├ [TX1] save(IN_PROGRESS) → COMMIT
        ├ [TX2] TransactionService.deposit()       ← @Transactional + @Retryable
        │         잔액 UPDATE + 이력 INSERT → COMMIT
        └ [TX3] complete(COMPLETED) → (실패하면) catch에서 키 삭제
```

트랜잭션이 3개로 쪼개져 있어서 TX2는 성공, TX3은 실패하는 상황이 생깁니다. 이게 R01의 원인입니다.

After

```
Controller
  └ IdempotencyService.execute(actorId, key, operation, canonicalRequest, ...)
        └ RetryTemplate (일시적 DB 충돌만, 최대 5회)        ← 재시도는 "트랜잭션 바깥"
              └ [TX 하나, REQUIRES_NEW, REPEATABLE_READ]
                    ├ 기존 키 있으면 → 지문 비교 → 재생 or 409
                    ├ saveAndFlush(IN_PROGRESS)  ← 유니크 제약이 동시 요청을 직렬화
                    ├ TransactionService.deposit(..., claim)  ← 같은 TX에 참여
                    │     잔액 UPDATE + 이력 INSERT(actor, 키 연결)
                    ├ claim.complete(응답 JSON)
                    └ COMMIT  → 이후 afterCommit에서만 성공 로그
```

키 선점, 잔액, 이력, 완료 저장이 **같이 커밋되거나 같이 롤백**됩니다.

2. 데이터 안전 (R01~R04)

R01 거래 커밋과 멱등성 완료 저장이 분리됨 → 중복 입금

> **문제** 시나리오 (리뷰 PDF D4, 실제 MySQL에서 재현)

1. 100원 입금 요청. 잔액 UPDATE와 이력 INSERT는 커밋됨 (잔액 100).
2. 이어서 키 상태를 `COMPLETED`로 바꾸는 UPDATE가 DB 오류로 실패.
3. `catch`가 거래 실패로 오해하고 **키를 삭제**. 클라이언트는 500을 받음.
4. 클라이언트가 같은 키로 재시도. 키가 없으니 다시 실행되고 **잔액 200, 이력 2건**이 됨.

> **BEFORE** `IdempotencyService.java` (7d6f766)

```
public <T> T execute(String key, Class<T> responseType, Supplier<T> action) {
    try {
        repository.save(new IdempotencyKey(key));          // TX1: 커밋됨
    } catch (DataIntegrityViolationException e) {
        return replayOrReject(key, responseType);
    }

    try {
        T result = action.get();                           // TX2: 서비스 프록시가 여기서 이미 커밋
        complete(key, result);                             // TX3: 여기서 실패하면?
        return result;
    } catch (RuntimeException e) {
        // action 실패와 complete 실패를 구분하지 못하고 키를 지움
        repository.findByKey(key).ifPresent(repository::delete);
        throw e;
    }
}
```

`TransactionService`에도 별도로 `@Transactional`과 `@Retryable`이 붙어 있어서, 서비스 메서드가 반환하는 순간 커밋이 끝났습니다.

> **AFTER** `IdempotencyService.java`

```
public IdempotencyService(IdempotencyKeyRepository repository, ObjectMapper objectMapper,
                          PlatformTransactionManager transactionManager) {
    // ...
    this.transaction = new TransactionTemplate(transactionManager);
    this.transaction.setPropagationBehavior(TransactionDefinition.PROPAGATION_REQUIRES_NEW);
    this.transaction.setIsolationLevel(TransactionDefinition.ISOLATION_REPEATABLE_READ);
}

private final RetryTemplate retry = RetryTemplate.builder().maxAttempts(5)
        .exponentialBackoff(50, 2, 400, true).retryOn(ConcurrencyFailureException.class).build();
```

```
public <T> T execute(Long actorId, String key, String operation, Object canonicalRequest,
                     Class<T> responseType, Function<IdempotencyKey, T> action) {
    // ... 입력 검증 (R18)
    String fingerprint = fingerprint(canonicalRequest);
    try {
        // 재시도가 커밋까지 감싼다 → rollback-only 트랜잭션을 재사용하는 시도가 없다
        return retry.execute(context -> executeAttempt(actorId, key, operation, fingerprint, responseType, acti
on));
    } catch (ConcurrencyFailureException ex) {
        throw new CustomException(ErrorCode.CONCURRENT_UPDATE_CONFLICT);
    }
}

private <T> T executeAttempt(...) {
    try {
        return transaction.execute(status -> {
            // ... 기존 키 조회 → 있으면 replay (R02)
            IdempotencyKey claim = repository.saveAndFlush(new IdempotencyKey(actorId, key, operation, fingerpr
int));
            T result = action.apply(claim);       // 잔액 변경 + 이력 저장이 같은 TX에서 실행
            claim.complete(serialize(result));    // 완료 저장도 같은 TX
            repository.flush();
            return result;                        // 여기까지 성공해야 한꺼번에 COMMIT
        });
    } catch (DataIntegrityViolationException ex) {
        if (!isClaimConflict(ex)) throw ex;
        // 유니크 충돌은 트랜잭션을 오염시키므로, 롤백된 뒤 새 트랜잭션에서 다시 조회
        return transaction.execute(status -> replay(repository.findByActorIdAndKey(actorId, key)
                .orElseThrow(...), operation, fingerprint, type));
    }
}
```

`TransactionService`에서는 `@Retryable`과 `@Recover` 메서드 4개를 모두 지웠습니다. 서비스는 호출자의 트랜잭션에 **참여만** 합니다.

```
// TransactionService.java (After)
// HTTP mutations join IdempotencyService's transaction. They never retry inside that transaction.
public TransactionResDto deposit(String accountNumber, Long requesterId, BigDecimal amount, IdempotencyKey clai
m) { ... }
```

왜 `execute()`에 `@Transactional`만 붙이지 않았나: 기존 `@Retryable`이 트랜잭션 **안쪽**에서 재시도하게 됩니다. 데드락으로 rollback-only가 된 트랜잭션 안에서 재시도하면 마지막 커밋이 `UnexpectedRollbackException`으로 실패합니다. 그래서 재시도(바깥)와 트랜잭션(안쪽)의 순서를 `RetryTemplate` → `TransactionTemplate`으로 코드에 직접 고정했습니다.

검증 `FinancialHardeningIntegrationTest.java` (실제 MySQL)

```java
@Test
void completionFailureRollsBackMoneyHistoryAndClaimThenRetryCommitsOnce() throws Exception {
    Actor a = actor();
    // COMPLETED로 바꾸는 UPDATE만 실패시키는 트리거 (리뷰 PDF D4와 같은 장애 주입)
    adminSql("CREATE TRIGGER test_fail_completion BEFORE UPDATE ON idempotency_key FOR EACH ROW "
            + "BEGIN IF NEW.status = 'COMPLETED' THEN SIGNAL SQLSTATE '45000' ...; END IF; END");
    try {
        request(a, depositPath(a), "completion-fault", "{\"amount\":100}")
                .andExpect(status().isInternalServerError());
        assertThat(balance(a)).isEqualByComparingTo("0");   // Before: 100 (커밋돼 버림)
        assertThat(history.count()).isZero();
        assertThat(keys.count()).isZero();
    } finally {
        adminSql("DROP TRIGGER IF EXISTS test_fail_completion");
    }
    request(a, depositPath(a), "completion-fault", "{\"amount\":100}").andExpect(status().isOk());
    assertThat(balance(a)).isEqualByComparingTo("100");     // Before: 200 (중복 반영)
    assertThat(history.count()).isEqualTo(1);
}
```

같은 파일의 다른 R01 관련 테스트:

- `transientMySqlFailureRetriesWholeTransactionIncludingClaim`: 첫 시도에 데드락(1213)을 주입하면 키 선점까지 포함해 **전체가 재시도**되고 1회만 반영되는지
- `serializationFailureRollsBackMoneyAndHistory`: 응답 직렬화가 실패하면 잔액·이력도 롤백되는지
- `ledgerInsertFailureRollsBackBalancesAndKey`: 이력 INSERT가 실패하면 잔액·키도 롤백되는지
- `repeatedTransientFailureReturnsConflictAndLeavesNoPartialEffects`: 재시도를 다 써도 부분 반영이 남지 않는지

R02 멱등성 키에 사용자·작업·요청 내용이 묶이지 않음

문제 시나리오 (리뷰 PDF D2, D3)

- **D2.** A가 키 `K`로 입금 → B가 같은 `K`로 A 계좌 출금 요청 → **200 OK와 함께 A의 계좌번호·잔액이 노출**됨. 기존 키를 찾으면 action을 실행하지 않으니 소유자 검증도 건너뛰었습니다.
- **D3.** 같은 사용자가 `K`로 100 입금 후 `K`로 200 입금 → 200 OK가 오지만 실제로는 100짜리 옛 응답. 클라이언트는 200이 들어간 줄 압니다.

BEFORE

```
// IdempotencyKey.java — key 하나만 전역 유니크
@Column(name = "idempotency_key", nullable = false, unique = true, length = 100)
private String key;

// TransactionController.java — 누가, 무엇을, 얼마를 요청했는지 넘기지 않음
idempotencyService.execute(idempotencyKey, TransactionResDto.class,
        () -> transactionService.deposit(accountNumber, requesterId, request.amount()));
```

AFTER

```
// IdempotencyKey.java — (사용자, 키) 조합이 유니크. 작업 종류와 요청 지문도 저장
@Table(uniqueConstraints = @UniqueConstraint(name = "uk_idempotency_actor_key",
        columnNames = {"actor_id", "idempotency_key"}), ...)
public class IdempotencyKey {
    @Column(name = "actor_id", nullable = false)            private Long actorId;
    @Column(name = "operation", nullable = false)           private String operation;          // DEPOSIT / WIT
HDRAW / TRANSFER
    @Column(name = "request_fingerprint", nullable = false) private String requestFingerprint; // SHA-256
    // ...
}
```

```
// TransactionController.java — 인증 주체, 작업, 정규화된 요청 내용을 함께 전달
TransactionResDto result = idempotencyService.execute(requesterId, idempotencyKey, "DEPOSIT",
        List.of(accountNumber, MoneyAmount.requireValid(request.amount())),   // ← 지문 원본
        TransactionResDto.class,
        claim -> transactionService.deposit(accountNumber, requesterId, request.amount(), claim));

// 이체는 수취 계좌까지 지문에 포함
List.of(request.fromAccountNumber(), request.toAccountNumber(), MoneyAmount.requireValid(request.amount()))
```

```
// IdempotencyService.java — 재생 전에 작업과 지문이 같은지 확인
private <T> T replay(IdempotencyKey claim, String operation, String fingerprint, Class<T> type) {
    if (!claim.getOperation().equals(operation) || !claim.getRequestFingerprint().equals(fingerprint)) {
        throw new CustomException(ErrorCode.IDEMPOTENCY_KEY_REUSED);   // 409 / C017
    }
    if (claim.getStatus() != IdempotencyStatus.COMPLETED) {
        throw new CustomException(ErrorCode.IDEMPOTENCY_REQUEST_IN_PROGRESS);
    }
    return objectMapper.readValue(claim.getResponseBody(), type);
}
```

포인트:

- `MoneyAmount.requireValid()`가 `setScale(2)`로 정규화하기 때문에 `100`과 `100.00`은 같은 지문이 되어 정상 재생됩니다 (`equivalentAmountReplaysSameTransaction...` 테스트).
- 사용자별로 키 공간이 분리되므로 B는 A의 키 문자열을 써도 A의 레코드를 볼 수 없습니다. B의 요청은 B 자신의 새 요청으로 처리되고, A 계좌 출금은 소유자 검증에서 403이 납니다.
- 마이그레이션 이전 키는 `actor_id = 0`, `legacy_key`로 격리합니다. 누가 보냈는지 모르는 옛 키는 재실행하지 않고 409로 거절합니다 (`V3__atomic_request_schema.sql`).

검증

```
@Test
void anotherActorCannotReplayOwnersResponseButCanUseSameKeyForOwnAccount() throws Exception {
    request(a, depositPath(a), "shared-key", "{\"amount\":100}").andExpect(status().isOk());
    request(b, "/api/accounts/" + a.account() + "/withdraw", "shared-key", "{\"amount\":1}")
            .andExpect(status().isForbidden());                // Before: 200 + A의 응답 노출
    request(b, depositPath(b), "shared-key", "{\"amount\":50}")
            .andExpect(status().isOk()).andExpect(jsonPath("$.data.accountNumber").value(b.account()));
}

@Test
void sameActorKeyRejectsChangedAmountOperationAndAccount() throws Exception {
    request(a, depositPath(a), "bound-key", "{\"amount\":100}").andExpect(status().isOk());
    request(a, depositPath(a), "bound-key", "{\"amount\":200}").andExpect(status().isConflict()); // Before: 20
0
    request(a, ".../withdraw", "bound-key", "{\"amount\":100}").andExpect(status().isConflict());
    // ...
}
```

`transferKeyCannotBeReusedForAnotherRecipient`: 같은 키로 수취인만 바꾸면 409.

R03 금액 소수 자릿수·상한 검증 없음 → 총액 보존 위반

문제 시나리오 (리뷰 PDF D1, MySQL 8.0 실측)

A=1.00, B=0.00에서 **0.015** 이체 → HTTP 200.

- A: 1.00 - 0.015 = 0.985 → DB `DECIMAL(19,2)` 반올림 → **0.99**
- B: 0.00 + 0.015 = 0.015 → 반올림 → **0.02**
- 합계 **1.01**. 돈이 0.01 생겼습니다. 응답 금액은 0.015, 이력에 저장된 금액은 0.02로 서로 달랐습니다.

BEFORE

```
// DepositReqDto / WithdrawReqDto / TransferReqDto
@DecimalMin(value = "0.01")   // 하한만 검사. 0.015는 0.01보다 크므로 통과
BigDecimal amount

// Account.java — DB 컬럼은 소수 2자리
@Column(name = "balance", precision = 19, scale = 2)
private BigDecimal balance;
```

AFTER 새 파일 `domain/account/MoneyAmount.java`

```
/** This simulator has one currency with two fractional digits; it never rounds customer input. */
public final class MoneyAmount {
    public static final BigDecimal MAX = new BigDecimal("9999999999999999.99");  // DECIMAL(19,2) 최댓값

    public static BigDecimal requireValid(BigDecimal amount) {
        if (amount == null || amount.signum() <= 0 || amount.scale() > 2 || amount.compareTo(MAX) > 0) {
            throw new CustomException(ErrorCode.INVALID_INPUT_VALUE);
        }
        return amount.setScale(2);
    }
}
```

이 검증을 **세 겹**으로 적용했습니다.

위치	코드	이유
HTTP DTO	`@Digits(integer = 17, fraction = 2)`	잘못된 요청을 400으로 빨리 거절
서비스	`amount = MoneyAmount.requireValid(amount);` (deposit/withdraw/transfer 첫 줄)	배치나 내부 호출처럼 MVC 검증을 거치지 않는 경로 보호
엔티티	`Account.deposit/withdraw`, `TransactionHistory` 생성자	도메인 객체가 잘못된 금액을 들고 있지 않도록

잔액 상한도 조건부 UPDATE에 넣었습니다.

```
// AccountRepository.java (After)
@Query("""
        update Account a
        set a.balance = a.balance + :amount, a.version = a.version + 1
        where a.id = :accountId
          and a.balance <= :maximumBalance - :amount      // ← 추가: 상한 초과 시 0건 업데이트
        """)
int increaseBalanceWithinLimit(Long accountId, BigDecimal amount, BigDecimal maximumBalance);

// TransactionService.java (After)
private void increaseBalance(Account account, BigDecimal amount) {
    if (accountRepository.increaseBalance(account.getId(), amount) != 1) {
        throw new CustomException(ErrorCode.BALANCE_LIMIT_EXCEEDED);   // 409 / C018
    }
}
```

이체 중 수취 계좌가 상한에 걸리면 예외가 나고, 이미 차감된 출금 쪽도 같은 트랜잭션이라 함께 롤백됩니다.

> **검증**

```
@Test
void fractionalCentTransferIsRejectedWithoutChangingTotalOrLedger() throws Exception {
    transactions.deposit(a.account(), a.id(), new BigDecimal("1.00"));
    request(a, "/api/transfer", "fraction", ... new BigDecimal("0.015") ...)
            .andExpect(status().isBadRequest());               // Before: 200
    assertThat(balance(a)).isEqualByComparingTo("1.00");       // Before: 0.99
    assertThat(balance(b)).isEqualByComparingTo("0.00");       // Before: 0.02
}

@Test
void serviceRejectsNegativeAndFractionalAmountsWithoutHttpValidation() {
    // HTTP를 거치지 않고 서비스를 직접 호출해도 거절되는지
    for (String text : new String[]{"-1", "0", "0.001", "0.015", "1.999", "100000000000000000"}) { ... }
}

@Test
void maximumBalanceRejectsOverflowAndRollsBackTransferDebit() { ... }  // 상한 초과 이체 시 출금도 롤백
```

R04 일반 사용자가 입금 API로 잔액을 만들어 냄

문제 회원가입 → 계좌 개설 → 입금 API에 원하는 금액 → 다른 계좌로 이체. 외부에서 돈이 들어온 증빙 없이 잔액이 생깁니다. 학습용 시뮬레이터로는 괜찮지만, 실제 돈과 연결되면 무담보 잔액이 됩니다.

BEFORE 소유자이기만 하면 입금 가능

```
public TransactionResDto deposit(String accountNumber, Long requesterId, BigDecimal amount) {
    Account account = accountService.getAccount(accountNumber);
    accountService.validateOwner(account, requesterId);   // 소유 확인 ≠ 입금 증빙
    increaseBalance(account, amount);
    // ...
}
```

AFTER

```
// TransactionService.java
@Value("${ledger.simulated-deposits-enabled:false}")      // 기본값 OFF
private boolean simulatedDepositsEnabled;

public TransactionResDto deposit(String accountNumber, Long requesterId, BigDecimal amount, IdempotencyKey clai
m) {
    if (!simulatedDepositsEnabled) throw new CustomException(ErrorCode.SIMULATED_DEPOSIT_DISABLED);  // 403 / C
019
    // ...
}
```

```
# application-prod.yaml — 운영은 무조건 OFF
ledger:
  simulated-deposits-enabled: false

# application-dev.yaml — 개발에서 환경변수로 켤 때만 ON
ledger:
  simulated-deposits-enabled: ${SIMULATED_DEPOSITS_ENABLED:false}
```

검증 `SimulatedDepositDisabledTest`, `ProductionBoundaryIntegrationTest`. prod 프로필에서 인증된 소유자가 입금해도 403/C019가 나고 잔액은 0으로 유지됩니다.

남은 범위: 실제 입금(외부 거래번호, 승인 증빙)을 연동하는 일은 이번 수정에 포함하지 않았습니다.

3. 보안·배포 기본 (R05~R09)

R05 검증 실패 로그에 비밀번호 평문

문제 비밀번호를 7자로 회원가입하면 `@Size(min = 8)` 검증이 실패합니다. 이때 `e.getMessage()`에 `rejected value [probe7]`이 들어 있어 WARN 로그로 그대로 나갔습니다.

BEFORE

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ApiResponse<Void>> handleValidation(MethodArgumentNotValidException e) {
    log.warn("Validation failed: {}", e.getMessage());   // ← rejected value 포함
    // ...
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ApiResponse<Void>> handleException(Exception e) {
    log.error("Unexpected error occurred", e);           // ← 스택/원인 메시지에 SQL 값이 섞일 수 있음
}
```

AFTER

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ApiResponse<Void>> handleValidation(MethodArgumentNotValidException e) {
    // Binding/JSON exceptions can contain passwords, tokens and complete rejected values.
    // Keep only server-selected categories and codes; requestId is already supplied by MDC.
    log.warn("Request rejected: code={} category=validation", ErrorCode.INVALID_INPUT_VALUE.getCode());
    // ...
}

@ExceptionHandler(Exception.class)
public ResponseEntity<ApiResponse<Void>> handleException(Exception e) {
    log.error("Unexpected error: code={} type={}", ErrorCode.INTERNAL_SERVER_ERROR.getCode(), e.getClass().getS
impleName());
}
```

나머지 핸들러(`HttpMessageNotReadable`, `TypeMismatch`, `MissingHeader`, `CustomException` 등)도 같은 방식으로 바꿨습니다. 로그는 코드 값을 직접 넣는 대신 **허용된 정보만** 고르는 방식입니다.

Hibernate가 직접 찍는 로그도 막았습니다.

```yaml
# application-common.yaml
logging:
  level:
    org.hibernate.SQL: warn                    # Before: debug
    org.hibernate.orm.jdbc.bind: off           # 바인드 파라미터(계좌번호·금액)
    # 중복 이메일/키 같은 드라이버 원문 메시지를 ERROR로도 내보내는 로거
    org.hibernate.engine.jdbc.spi.SqlExceptionHelper: off
```

같은 맥락에서 `LoginReqDto`, `MemberCreateReqDto`에 길이 상한을 추가했습니다 (이메일·이름 255자, 비밀번호 BCrypt 한계인 UTF-8 72바이트).

> **검증** `ExceptionLoggingSafetyTest` . 잘못된 DTO·JWT 입력과 실제 MySQL 중복 INSERT를 발생시킨 뒤, 루트 로그와 예외 스택 전체에 민감값이 없는지 확인합니다.

R06 `X-Forwarded-For`를 그대로 믿어 로그인 제한 우회

문제 클라이언트가 요청마다 `X-Forwarded-For: 1.1.1.1`, `1.1.1.2`, …처럼 헤더를 바꾸면 매번 새 버킷을 받습니다. 로그인 10회 제한이 의미가 없어집니다.

BEFORE

```
String resolveClientKey(HttpServletRequest request) {
    String forwardedFor = request.getHeader("X-Forwarded-For");
    if (forwardedFor != null && !forwardedFor.isBlank()) {
        return forwardedFor.split(",")[0].trim();   // 클라이언트가 보낸 값을 그대로 키로 사용
    }
    return request.getRemoteAddr();
}
```

AFTER

```
// Forwarded headers are untrusted input. Keep server.forward-headers-strategy=none:
// NAT/proxy clients deliberately share a bucket until a trusted edge enforces per-client limits.
String resolveClientKey(HttpServletRequest request) {
    return request.getRemoteAddr();
}
```

```
# application-common.yaml — Spring이 Forwarded 헤더로 remoteAddr를 바꾸지 않게
server:
  forward-headers-strategy: none
```

트레이드오프: 프록시나 NAT 뒤의 사용자들은 버킷 하나를 같이 씁니다. 고객별로 정확히 제한하려면 신뢰할 수 있는 경계 장비(Ingress)가 헤더를 정규화해 주거나, 계정 단위 제한을 추가해야 합니다.

검증 `RateLimitSecurityTest.differentForwardedHeadersShareTheSocketPeersLoginBudget`. `X-Forwarded-For`와 `Forwarded`를 각각 다르게 보내도 같은 버킷이 소비됩니다.

R07 Redis 연결 누락 + 장애 시 제한 해제(fail-open)

문제 CD 시크릿에 Redis 주소가 없어 `${REDIS_HOST:localhost}` 기본값이 쓰였습니다. 파드 안에는 Redis가 없으니 매 요청이 실패하고, 실패하면 **무조건 허용**했습니다. 결과적으로 배포 환경에서는 rate limit이 꺼진 것과 같았습니다.

BEFORE

```
# application.yaml
data:
  redis:
    host: ${REDIS_HOST:localhost}
```

```
// TokenBucketRateLimiter.java
} catch (Exception e) {
    log.warn("Rate limit check failed, allowing request (fail-open): {}", e.getMessage());
    return true;   // 로그인 브루트포스 방어까지 같이 꺼짐
}
```

AFTER

```
# application-common.yaml — 기본값 없음: 주입 안 하면 기동 실패
data:
  redis:
    host: ${REDIS_HOST}
    password: ${REDIS_PASSWORD:}
    connect-timeout: 1s
    timeout: 1s

# application-prod.yaml
    ssl:
      enabled: ${REDIS_SSL_ENABLED:true}
```

```
// TokenBucketRateLimiter.java — fail-closed
} catch (DataAccessException e) {
    // Neither login nor general APIs silently lose protection during a Redis outage.
    // Exception text can include connection credentials; keep server-selected metadata only.
    log.warn("Rate limit unavailable: code={} type={}", ErrorCode.RATE_LIMIT_UNAVAILABLE.getCode(), e.getClass
().getSimpleName());
    throw new CustomException(ErrorCode.RATE_LIMIT_UNAVAILABLE);   // → 필터에서 503 + Retry-After
}
```

Lua 스크립트도 함께 고쳤습니다.

```
-- Before: 앱 서버 시계를 인자로 받음 → 파드마다 시계가 다르면 리필 계산이 어긋남
local now = tonumber(ARGV[4])
redis.call('EXPIRE', key, refillPeriodSeconds * 2)

-- After: Redis 서버 시계 하나만 사용, TTL은 "가득 찰 때까지 걸리는 시간"으로
local redisTime = redis.call('TIME')
local now = tonumber(redisTime[1]) + tonumber(redisTime[2]) / 1000000
redis.call('EXPIRE', key, math.max(1, math.ceil(capacity / refillTokens * refillPeriodSeconds)))
```

`RateLimitProperties`에는 0 이하 설정값을 기동 시점에 거절하는 검증을 넣었습니다.

트레이드오프: Redis가 죽으면 API가 503을 냅니다. 금융 API에서는 보호 장치 없이 계속 받는 것보다 잠시 거절하는 쪽을 택했습니다. 대신 readiness에 Redis를 포함해서(R16) 장애 파드로는 트래픽이 가지 않게 했습니다.

> **검증** `RedisTokenBucketContractTest` (실제 Redis에 50개 동시 요청 → 허용 정확히 5개, refill/TTL), `RateLimitSecurityTest` (장애 시 503).

R08 운영 스키마를 앱 기동 시 자동 변경 (`ddl-auto: update`)

문제 prod에서도 엔티티를 바꾸면 앱이 뜨면서 DDL을 실행했습니다. 변경 이력이 없고, 롤링 업데이트 중 테이블 잠금이 생길 수 있으며, 코드를 되돌려도 스키마는 돌아오지 않습니다.

BEFORE

```
spring:
  jpa:
    hibernate:
      ddl-auto: update
  # Spring Batch 메타테이블은 jar 안의 SQL로 "수동 1회" 생성
```

AFTER

```
src/main/resources/db/migration/
├── V1__original_business_schema.sql     ← 기존 운영 스키마를 그대로 기록 (baseline)
├── V2__spring_batch_5_2_schema.sql      ← 수동이던 BATCH_* 테이블
├── V3__atomic_request_schema.sql        ← R02/R10: actor_id, fingerprint, 이력-키 연결
└── V4__reconciliation_evidence.sql      ← R12: 대사 결과 테이블
```

```
# application-prod.yaml — 런타임 계정은 DML만, 스키마는 검증만
spring:
  flyway:
    enabled: false
  jpa:
    hibernate:
      ddl-auto: validate
```

마이그레이션은 앱과 분리된 명령으로 실행합니다.

```
// common/config/DatabaseMigration.java — 웹서버·스케줄러 없이 Flyway만 실행
public static void execute(String action, Map<String, String> environment) {
    if (!Arrays.asList("migrate", "validate", "baseline").contains(action)) {
        throw new IllegalArgumentException("... destructive clean is not supported");
    }
    if (action.equals("baseline") && !"original-schema-reviewed".equals(environment.get("FLYWAY_BASELINE_AC
K"))) {
        throw new IllegalArgumentException("Existing schema and backup must be reviewed before baseline");
    }
    // ... TLS URL 검증
    Flyway flyway = Flyway.configure()...
            .cleanDisabled(true).baselineOnMigrate(false)...;
}
```

배포 시에는 `k8s/migration-job.yaml`이 마이그레이션 전용 계정(`account-migration-secret`)으로 이 명령을 실행합니다. CD 순서는 앱 축소 → 마이그레이션 Job 완료 대기 → 새 앱 적용입니다.

검증 `SchemaMigrationTest` (빈 DB 설치, 재실행, 기존 DB baseline 후 데이터 보존), `DatabaseMigrationCommandTest` (clean 거부, baseline 확인값 없으면 거부, TLS 강제). 절차는 `OPERATIONS_RUNBOOK.md` 에 있습니다.

R09 Ingress에 HTTPS 강제 구성이 없음

BEFORE

```
spec:
  rules:
    - http:
        paths:
          - path: /            # host·tls 없음, 모든 경로 노출
```

AFTER `k8s/ingress.yaml`

```
metadata:
  annotations:
    traefik.ingress.kubernetes.io/router.entrypoints: websecure   # 443만
    traefik.ingress.kubernetes.io/router.tls: "true"
spec:
  ingressClassName: traefik
  tls:
    - hosts: [__PUBLIC_HOST__]          # CD에서 render_manifests.py가 치환
      secretName: account-tls
  rules:
    - host: __PUBLIC_HOST__
      http:
        paths:
          - path: /api                  # 업무 API만 외부 노출
```

새 파일 `k8s/networkpolicy.yaml`: 8080은 Traefik에서 오는 요청만, 9090(관리 포트)은 `monitoring` 네임스페이스에서 오는 요청만 허용합니다.

한계: YAML 문법과 템플릿 렌더링까지만 검증했습니다. 실제 인증서, Traefik 라벨, CNI의 NetworkPolicy 지원 여부는 클러스터에서 확인해야 합니다.

4. 운영 신뢰 (R10~R19)

R10 커밋 전 성공 로그 + 감사 연결 정보 부족

문제 `log.info("이체 완료 ...")` 가 메서드 **안에서** 실행되고 커밋은 메서드가 반환된 **뒤에** 일어납니다. 커밋이 실패해도 "완료" 로그는 이미 남아 있습니다. 또 이력에 "누가, 어떤 요청으로" 했는지가 없었습니다.

BEFORE

```
TransactionHistory history = transactionHistoryRepository.save(new TransactionHistory(from, to, amount, TRANSFE
R));
log.info("이체 완료: from={}, to={}, amount={}", mask(from), mask(to), amount);   // 커밋 전
return new TransferResDto(...);                                                   // 프록시가 이 뒤에 커밋
```

AFTER

```
// TransactionService.java
private TransactionHistory record(Account from, Account to, BigDecimal amount, TransactionType type,
                                  Long actorId, IdempotencyKey claim) {
    TransactionHistory history = transactionHistoryRepository.save(
            new TransactionHistory(from, to, amount, type, actorId, claim));   // 수행자 + 요청 키 연결
    if (TransactionSynchronizationManager.isSynchronizationActive()) {
        Long transactionId = history.getId();
        Long idempotencyId = history.getIdempotencyId();
        TransactionSynchronizationManager.registerSynchronization(new TransactionSynchronization() {
            @Override
            public void afterCommit() {   // 커밋이 실제로 성공했을 때만 실행
                log.info("Transaction committed: transactionId={}, actorId={}, operation={}, idempotencyId={}",
                        transactionId, actorId, type, idempotencyId);
            }
        });
    }
    return history;
}
```

```
// TransactionHistory.java — 컬럼 추가
@Column(name = "actor_id")                 private Long actorId;
@ManyToOne @JoinColumn(name = "idempotency_key_id") private IdempotencyKey idempotencyKey;

// 응답 DTO에 추적 ID 추가
public record TransactionResDto(String accountNumber, BigDecimal balance, BigDecimal amount,
                                TransactionType type, LocalDateTime createdAt,
                                Long transactionId, Long actorId, Long idempotencyId) {}
```

성공 로그에서 계좌번호와 금액을 아예 뺐습니다. 내부 ID만으로 원장(DB)을 찾아갈 수 있습니다.

검증 `failedCommitEmitsNoSuccessfulTransactionLog` (커밋 실패 시 INFO 로그 0건), `successfulAuditLogContainsCommittedIdsWithoutAccountOrAmount`.

R11 대사 배치의 `businessDate`가 실제 집계 기준과 무관

문제 파라미터 이름은 "전일 영업일"인데 쿼리에는 날짜 조건이 없습니다. 실제로는 **지금 시점의 잔액**과 **전체 기간 이력 합계**를 비교했습니다. 일 마감처럼 보이지만 그렇지 않았습니다.

수정 방향: 일 마감을 새로 구현하지 않고, 이름과 의미를 실제 동작에 맞췄습니다. 리뷰 PDF가 경고한 "쿼리에 날짜 조건만 붙여서 현재 잔액과 비교하는 수정"은 하지 않았습니다.

```
// Before
LocalDate businessDate = LocalDate.now(ZoneId.of(ZONE)).minusDays(1);
new JobParametersBuilder().addLocalDate("businessDate", businessDate)

// After — "언제 돌렸는지"일 뿐, 원장 cutoff가 아님을 이름으로 드러냄
new JobParametersBuilder().addLocalDate("scheduleDate", now.toLocalDate())
```

```
// BalanceReconciliationJobConfig.java
/** Current consistency scan, not a historical closing balance or business-day settlement. */

// 청크마다 계좌 조회와 이력 SUM이 같은 MySQL 스냅샷을 보도록 격리 수준을 명시
transaction.setIsolationLevel(TransactionDefinition.ISOLATION_REPEATABLE_READ);
```

결과 테이블에도 `basis = 'CURRENT_LEDGER_CONSISTENCY'`를 기록합니다 (R12).

검증 계좌를 조회한 뒤 입금이 커밋되어도, 같은 청크의 스냅샷으로 비교하기 때문에 정상 계좌를 불일치로 잘못 판정하지 않습니다.

R12 대사 불일치가 로그로만 남음

BEFORE

```
private ItemWriter<BalanceMismatch> mismatchLogWriter() {
    return chunk -> chunk.getItems().forEach(mismatch ->
            log.warn("잔고 불일치 발견: accountId={}, account={}, 차이={}", ...));   // 끝. Job은 COMPLETED
}
```

AFTER 새 파일 `ReconciliationResults.java` + `V4__reconciliation_evidence.sql`

```
CREATE TABLE reconciliation_run (       -- 실행 1회 = 1행
    id BIGINT PRIMARY KEY,              -- = Batch JobExecution id
    schedule_date DATE, basis VARCHAR(48), status VARCHAR(16), mismatch_count BIGINT, ...
);
CREATE TABLE reconciliation_mismatch (  -- 불일치 계좌 1개 = 1행
    run_id BIGINT, account_id BIGINT, actual_balance DECIMAL(19,2), expected_balance DECIMAL(19,2),
    resolution_status VARCHAR(16) DEFAULT 'OPEN',          -- 조사 상태 관리
    CONSTRAINT uk_reconciliation_run_account UNIQUE(run_id, account_id)
);
```

```
@Transactional
public void write(Long runId, Chunk<? extends BalanceMismatch> chunk) {
    // Unique key plus no-op upsert preserves original evidence when a chunk is replayed.
    jdbc.batchUpdate("""
            INSERT INTO reconciliation_mismatch (...) VALUES (?, ?, ?, ?, ?, 'OPEN')
            ON DUPLICATE KEY UPDATE id = id
            """, ...);
}
```

DB 기반 지표를 등록해서 재시작하거나 파드가 여러 개여도 같은 값이 나옵니다.

```
registry.gauge("reconciliation.mismatches", this, r -> r.latestMismatchCount());
registry.gauge("reconciliation.last.completed.timestamp", this, r -> r.latestTimestamp(false));
registry.gauge("reconciliation.runs.failed", ...);
```

```
# k8s/prometheusrule.yaml — 추가된 경보
- alert: AccountLedgerMismatch
  expr: max(reconciliation_mismatches{job="account-service"}) > 0
- alert: AccountReconciliationStale          # 27시간 동안 완료된 대사가 없음
  expr: time() - max(reconciliation_last_completed_timestamp{...}) > 97200 or absent(...)
- alert: AccountReconciliationFailure
```

잔액을 자동으로 고치지는 않습니다. 이 원칙은 기존 그대로 유지했습니다.

R13 배치 누락·중단 후 복구 경로 없음

문제

- `@Scheduled(cron = "0 0 1 * * *")`는 01:00 정각 한 번만 호출합니다. 그 순간 앱이 꺼져 있으면 그날은 건너뜁니다.
- 실행 중에 파드가 죽으면 `STARTED` 상태가 남고, 같은 날짜는 "이미 실행 중"으로 막힙니다.

AFTER `BalanceReconciliationScheduler.java`

```
// Polling also catches application startup after 01:00 and retries FAILED executions.
@Scheduled(fixedDelayString = "${reconciliation.scheduler.poll-delay-ms:300000}")   // 5분마다
public void runDailyReconciliation() {
    LocalDateTime now = LocalDateTime.ofInstant(clock.instant(), ZONE);
    if (now.toLocalTime().isBefore(LocalTime.of(1, 0))) return;
    try {
        // 같은 날짜 = 같은 JobInstance → 이미 완료된 날은 Batch가 막아줌, FAILED는 재시작됨
        JobExecution execution = launcher.run(job, new JobParametersBuilder()
                .addLocalDate("scheduleDate", now.toLocalDate()).toJobParameters());
        // ...
    } catch (JobInstanceAlreadyCompleteException | JobExecutionAlreadyRunningException normalOverlap) {
        // Never time out or reset STARTED: an old timestamp cannot prove the worker is dead.
    } catch (Exception failure) {
        launchFailures.increment();   // → AccountReconciliationFailure 경보
    }
}
```

고착된 `STARTED`는 **자동으로 정리하지 않습니다.** 오래됐다는 것만으로는 워커가 죽었다고 단정할 수 없기 때문입니다. 대신 모든 실행기를 멈춘 것을 확인한 뒤 운영자가 실행하는 스크립트를 두었습니다.

```
-- scripts/recover-reconciliation.sql (recover-reconciliation.ps1로만 실행)
-- The caller sets @execution_id to a positive integer. Never infer death from elapsed time.
UPDATE BATCH_JOB_EXECUTION SET STATUS = 'FAILED', ...
WHERE JOB_EXECUTION_ID = @eligible AND STATUS IN ('STARTING', 'STARTED', 'STOPPING');
```

`FAILED`로 바꿔면 다음 폴링에서 자동으로 재실행됩니다.

검증 `BalanceReconciliationSchedulerIntegrationTest` . 01시 이후 늦게 기동한 경우, FAILED 재시도, 완료된 날 중복 실행 방지, 살아 있는 STARTED 보호, 복구 SQL 동작을 확인합니다.

R14 테스트를 통과하지 않은 커밋도 배포됨

BEFORE

```
# ci.yml
on:
  pull_request:          # PR에서만 테스트

# cd.yml
on:
  push:
    branches: [master]   # 테스트와 무관하게 빌드.배포
# Dockerfile 안에서 bootJar만 실행 (test 없음)
```

AFTER

```
# ci.yml
on:
  pull_request:
  workflow_call:          # CD에서 재사용 가능하게

# cd.yml
jobs:
  test:
    uses: ./.github/workflows/ci.yml   # 같은 SHA로 전체 테스트
  build:
    needs: test                        # 테스트 성공해야 빌드
    # ./gradlew bootJar → docker build (Dockerfile은 빌드된 jar만 복사)
  deploy:
    if: github.event_name == 'workflow_dispatch' && inputs.maintenance_window   # 수동 + 점검창 확인
    needs: build
    environment: production
```

Dockerfile은 빌드 단계를 없애고, CI가 검증한 jar만 non-root 사용자(uid 10001)로 실행합니다.

한계: GitHub 브랜치 보호 규칙과 실제 워크플로 실행 이력은 확인하지 않았습니다.

R15 `latest` 적용 후 SHA로 바꾸는 이중 롤아웃 + 동시 배포

BEFORE

```
- run: kubectl apply -f k8s/deployment.yaml          # image: ...:latest → 롤아웃 1
- run: kubectl set image ... :${{ github.sha }}      # → 롤아웃 2
# concurrency 설정 없음 → 늦게 끝난 옛 배포가 새 버전을 덮어쓸 수 있음
```

AFTER

```
concurrency:
  group: account-production-release
  cancel-in-progress: false            # 배포 직렬화

# build: push 후 불변 digest 확보
digest=$(aws ecr describe-images ... --query 'imageDetails[0].imageDigest' --output text)
echo "image=$ECR_REPOSITORY@$digest" >> "$GITHUB_OUTPUT"

# deploy: 최신 master인지 확인 → digest로 렌더링 → 한 번만 apply
test "$(git ls-remote origin refs/heads/master | cut -f1)" = "$GITHUB_SHA"
python3 scripts/render_manifests.py --output release.yaml
kubectl apply -f release.yaml
```

```
# scripts/render_manifests.py — 태그(latest, sha) 대신 digest만 허용
if not re.fullmatch(r'[a-zA-Z0-9][a-zA-Z0-9./:_-]*@sha256:[a-f0-9]{64}', image):
    raise ValueError('ACCOUNT_IMAGE must be an immutable registry/repository@sha256 digest')
```

`k8s/deployment.yaml`의 이미지는 `__IMAGE__` 플레이스홀더로 바뀌었습니다. 치환되지 않은 값이 남아 있으면 렌더러가 실패합니다.

검증 `scripts/test_render_manifests.py` (CI에서 실행). 변경 가능한 태그와 잘못된 호스트를 거절하는지, 정상 digest가 렌더링되는지 확인합니다.

R16 관리 포트만 헬스체크 → 실제 API 장애를 놓침

BEFORE

```
readinessProbe:
  httpGet: { path: /actuator/health/readiness, port: 9090 }
livenessProbe:
  httpGet: { path: /actuator/health/liveness, port: 9090 }
```

9090(관리 서버)은 살아 있는데 8080(API 서버)의 스레드가 고갈되면, 파드가 계속 Ready로 남아 트래픽을 받습니다.

AFTER

```
# application-common.yaml
management:
  endpoint:
    health:
      probes:
        add-additional-paths: true        # 주 포트(8080)에 /livez, /readyz 노출
      group:
        readiness:
          include: readinessState,db,redis   # 트래픽 받을 수 있나 = DB·Redis까지
        liveness:
          include: livenessState             # 재시작이 필요한가 = 프로세스 자체만

# k8s/deployment.yaml
startupProbe:   { httpGet: { path: /livez,  port: http } }   # 기동 최대 150초 허용
readinessProbe: { httpGet: { path: /readyz, port: http } }
livenessProbe:  { httpGet: { path: /livez,  port: http } }
```

왜 liveness에 DB를 넣지 않았나: DB 하나가 잠깐 느려지면 모든 파드가 동시에 재시작되는 "재시작 폭풍"이 납니다. DB 장애는 readiness로 트래픽만 빼는 것이 맞습니다.

보안 설정(`SecurityConfig`)도 함께 바꿨습니다. `/livez`와 `/readyz`만 익명으로 허용하고(상세 정보는 비노출), `/actuator/prometheus`는 관리 포트로 들어온 GET일 때만 허용합니다. Swagger는 `app.security.public-docs=true`일 때만 열립니다.

검증 `SecurityBoundaryTest`, `ProductionBoundaryIntegrationTest`. 실제 HTTP로 프로브가 200을 반환하고 세부 정보는 노출하지 않는지, 그 밖의 관리 경로와 Swagger는 익명 요청에 401을 반환하는지 확인합니다.

R17 동시 이체 테스트가 전부 실패해도 통과

문제 양방향 이체 100건이 **모두** `CONCURRENT_UPDATE_CONFLICT`로 실패해도, 잔액이 그대로라 "합계 보존" 단언은 통과합니다. 테스트가 버그를 잡지 못하는 구조였습니다.

BEFORE

```
} catch (CustomException e) {
    if (e.getErrorCode() != ErrorCode.CONCURRENT_UPDATE_CONFLICT) {
        unexpectedFailures.add(e);           // 충돌 실패는 "예상된 실패"로 무시
    }
}
// ...
assertThat(balanceA.add(balanceB)).isEqualByComparingTo(BigDecimal.valueOf(200_000));   // 이것만 검증
```

AFTER

```
} catch (CustomException e) {
    unexpectedFailures.add(e);               // 어떤 실패든 실패로 집계
}
// ...
assertThat(balanceA.add(balanceB)).isEqualByComparingTo(BigDecimal.valueOf(200_000));
assertThat(successes.get()).isEqualTo(100);          // 100건 모두 성공했는지
assertThat(history.count()).isEqualTo(102);          // 이력 = 초기 입금 2 + 이체 100
assertThat(balanceA).isEqualByComparingTo("100000"); // 양쪽 순변동이 정확한지
assertThat(balanceB).isEqualByComparingTo("100000");
```

추가로 H2를 쓰던 `MemberRepositoryTest`를 포함해 모든 DB 테스트를 MySQL Testcontainers + Flyway로 통일했습니다 (CLAUDE.md의 "H2 금지" 규칙).

R18 멱등성 헤더의 빈 값·길이·충돌 종류 미검증

문제

- 공백 키(`" "`)도 통과해서, 서로 다른 거래가 같은 키로 묶일 수 있었습니다.
- 101자 키는 DB 길이 초과 오류가 나는데, 이 오류도 `DataIntegrityViolationException` 이라 "중복 키"로 오인해 500이 났습니다.

BEFORE

```
try {
    repository.save(new IdempotencyKey(key));
} catch (DataIntegrityViolationException e) {   // 길이 초과, NOT NULL 위반 등 무엇이든 "중복"으로 처리
    return replayOrReject(key, responseType);
}
```

AFTER

```
// 업무 실행 전에 형식 검증 → 400
if (actorId == null || actorId <= 0 || key == null || !key.matches("[A-Za-z0-9._:-]{1,100}")) {
    throw new CustomException(ErrorCode.INVALID_INPUT_VALUE);
}

// MySQL 오류 1062 + 우리가 이름 붙인 제약일 때만 "중복 요청"으로 판단
static boolean isClaimConflict(Throwable error) {
    for (Throwable cause = error; cause != null; cause = cause.getCause()) {
        if (cause instanceof ConstraintViolationException violation
                && violation.getErrorCode() == 1062 && "23000".equals(violation.getSQLState())) {
            String constraint = violation.getConstraintName();
            return constraint != null && (constraint.equals("uk_idempotency_actor_key")
                    || constraint.endsWith(".uk_idempotency_actor_key"));
        }
    }
    return false;
}
```

새 키 컬럼은 `utf8mb4_0900_bin` 콜레이션이라 대소문자와 후행 공백을 구분합니다. 기존 키는 `legacy_key` 에 원래 콜레이션 그대로 보존했습니다.

검증 `malformedKeysAreRejectedBeforeClaimOrBalanceChange` (`""` , `" "` , 101자, 공백 포함, `/` , 한글 → 모두 400, 키 저장 0건), `IdempotencyConstraintTest` .

남은 범위: 키 보존 기간과 정리 정책은 아직 정하지 않았습니다.

R19 거래 이력 페이지 정렬 기준이 고유하지 않음

문제 `ORDER BY createdAt DESC` 만 있어서, 같은 시각에 여러 건이 있으면 페이지 경계에서 순서가 바뀔 수 있습니다. 또 1페이지를 본 뒤 새 거래가 생기면 offset이 밀려 2페이지에 이미 본 행이 다시 나옵니다.

BEFORE

```
@Query("... WHERE t.fromAccount = :account OR t.toAccount = :account ORDER BY t.createdAt DESC")
Page<TransactionHistory> findByAccount(Account account, Pageable pageable);   // 클라이언트 sort도 그대로 적용
```

AFTER

```
// 1) 고유한 id로 정렬 고정. 클라이언트가 보낸 sort는 무시하고 size는 1~100으로 제한
"... ORDER BY t.id DESC"

return transactionHistoryRepository.findByAccount(account,
        PageRequest.of(pageable.getPageNumber(), pageable.getPageSize()))...;   // sort 제거

// 2) 커서 API 추가: GET /api/accounts/{no}/transactions/cursor?beforeId=&size=
@Query("SELECT t FROM TransactionHistory t LEFT JOIN FETCH t.fromAccount LEFT JOIN FETCH t.toAccount "
        + "WHERE (t.fromAccount = :account OR t.toAccount = :account) "
        + "AND (:beforeId IS NULL OR t.id < :beforeId) ORDER BY t.id DESC")
List<TransactionHistory> findByAccountBeforeId(Account account, Long beforeId, Pageable pageable);
```

```
// TransactionService.getHistoryCursor — size+1개를 읽어 다음 페이지 존재 여부 판단
var rows = repository.findByAccountBeforeId(account, beforeId, PageRequest.of(0, size + 1));
boolean hasNext = rows.size() > size;
Long nextCursor = hasNext ? items.get(items.size() - 1).id() : null;
```

`(from_account_id, id)`, `(to_account_id, id)` 복합 인덱스도 추가했습니다. `fetch join`은 그대로 유지해서 N+1이 생기지 않습니다.

검증 `historyWithEqualTimestampsUsesDescendingUniqueIds` (같은 timestamp 3건 → id 큰 순서), `cursorHistoryDoesNotRepeatRowsWhenNewTransactionsArrive` (1페이지 조회 후 새 거래가 생겨도 다음 커서 페이지에 중복 없음).

5. 차단 이슈 외 함께 고친 것 (리뷰 PDF 10장)

항목	수정
문서와 실제 동시성 방식 불일치	`Account`의 `@Version` 주석을 "조건부 원자적 UPDATE가 주 경로, @Version은 엔티티 갱신 경로 보호용"으로 수정. 과거 측정은 `HISTORICAL_CONCURRENCY_NOTES.md`로 이동
도메인 입력 방어	`Account.deposit/withdraw`에 `MoneyAmount.requireValid` (R03)
JWT 이중 파싱	`validateToken()` + `getMemberId()` 두 번 파싱하던 것을 `getMemberId()` 한 번으로 합침. 실패 시 `null` 반환, 만료 없는 토큰과 0 이하 subject도 거절
토큰 버킷 설정 검증	`RateLimitProperties`에 양수 검증 (R07)
OSIV	`spring.jpa.open-in-view: false`
컨테이너 보안	non-root, `readOnlyRootFilesystem`, capability 전부 drop, SA 토큰 미마운트

6. 아직 남은 것 (이번 수정 범위 밖)

- **실제 입금 연동** (R04): 외부 거래번호, 승인 증빙, 중복 외부 거래 차단
- **과거 일 마감** (R11): 기준 시점 스냅샷과 확정 거래 집계를 비교하는 별도 설계
- **멱등성 키 보존 기간** (R18): 업무상 재시도 기간에 맞춘 정리 정책
- **운영 환경 증거**: TLS 인증서, NetworkPolicy 실제 적용, DB 계정 권한, 경보 수신 경로, 백업/PITR 복원, 브랜치 보호
- **장시간 부하·프로세스 강제 종료** 같은 카오스 테스트 (`CHAOS_AND_LOAD_TEST_SPEC.md`)

7. API 호환성 주의

클라이언트에서 바뀌는 동작입니다. 자세한 내용은 `API_CHANGES.md`에 있습니다.

변경	이전	이후
소수 3자리 이상 금액	200 (DB 반올림)	400
같은 키, 다른 요청	200 (옛 응답)	409 / C017
멱등성 키 형식	아무 문자열	`[A-Za-z0-9._:-]{1,100}`
입금 API	항상 가능	prod 403 / C019
Redis 장애	요청 허용	503 / C020 + `Retry-After`
거래 응답	-	`transactionId`, `actorId`, `idempotencyId` 필드 추가
이력 정렬	`createdAt DESC`, `sort` 파라미터 반영	`id DESC` 고정, `size` 1~100